

6. Автоэнкодеры

Цель работы - реализовать и исследовать автоэнкодеры с полносвязными слоями для удаления шума с изображения. В качестве выборки используется MNIST. В рамках работы требуется реализовать несколько вариантов зашумления, подобрать параметры архитектуры автоэнкодера и оценить качество восстановления изображений.

Задания:

1. Реализовать различные варианты зашумления и исследовать, как зависит качество нормализации изображения (очистки от шума):
 - а) от типа зашумления: равномерный шум, нормальный шум, локальная порча изображений («пятна» на изображении);
 - б) от размерности пространства скрытых признаков;
 - в) от количества и ширины слоев кодера и декодера;
 - г) от количества эпох;
 - д) от коэффициента обучения.
2. Оценить качество изображений, восстановленных из чистых и зашумленных изображений.
3. Оценить возможность генерации новых изображений из случайных точек в латентном пространстве (пространстве скрытых признаков).

Ссылка на исходный код лабораторной работы:

https://github.com/F1ameX/Modern-Methods-of-Deep-Machine-Learning/blob/main/6_autoencoders.ipynb

Ход работы

Реализация автоэнкодера с полносвязными слоями

В первой части работы был реализован автоэнкодер на базе модуля torch. Модель состоит из двух частей: кодера, переводящего изображение в пространство скрытых признаков, и декодера, восстанавливающего изображение по латентному вектору.

Изображение размера 28 x 28 предварительно разворачивается в вектор длины 784. В базовой конфигурации кодер имеет структуру 784 -> 512 -> 256 -> z, где z - латентное пространство заданной размерности. Декодер симметрично восстанавливает изображение по схеме z -> 256 -> 512 -> 784.

На выходе используется Sigmoid, так как значения пикселей нормированы в диапазон [0; 1].

```
class DenseAutoencoder(nn.Module):
    def __init__(self, input_dim=28 * 28,
                  hidden_dims=(512, 256), latent_dim=32):
        super().__init__()
        encoder_layers = []
        prev_dim = input_dim
        for hidden_dim in hidden_dims:
            encoder_layers += [nn.Linear(prev_dim, hidden_dim), nn.ReLU()]
            prev_dim = hidden_dim
        encoder_layers.append(nn.Linear(prev_dim, latent_dim))
        self.encoder = nn.Sequential(*encoder_layers)

        decoder_layers = []
        prev_dim = latent_dim
        for hidden_dim in reversed(hidden_dims):
            decoder_layers += [nn.Linear(prev_dim, hidden_dim), nn.ReLU()]
            prev_dim = hidden_dim
        decoder_layers += [nn.Linear(prev_dim, input_dim), nn.Sigmoid()]
        self.decoder = nn.Sequential(*decoder_layers)

    def encode(self, x):
        return self.encoder(x.view(x.size(0), -1))

    def decode(self, z):
        return self.decoder(z.view(-1, 1, 28, 28))

    def forward(self, x):
        return self.decode(self.encode(x))
```

Рисунок 1 - Реализация полносвязного автоэнкодера

Подготовка выборки и вариантов зашумления

Для обучения и проверки использована выборка MNIST. Метки классов в задаче восстановления не используются: входом модели является зашумленное изображение, а целевым значением - соответствующее чистое изображение.

В работе реализованы три типа зашумления, требуемые заданием. Равномерный и нормальный шум добавляются к значениям пикселей, после чего изображение обрезается к диапазону [0; 1]. Локальная порча реализована как наложение квадратных пятен на случайные участки изображения.

Таблица 1 - Варианты зашумления изображений

Тип зашумления	Способ формирования	Параметры в эксперименте
Равномерный шум	К пикселям добавляется шум из равномерного распределения	level = 0.45

Нормальный шум	К пикселям добавляется шум из нормального распределения	std = 0.35
Локальные пятна	На изображение накладываются квадратные области случайного значения	число пятен = 2, размер пятна = 7

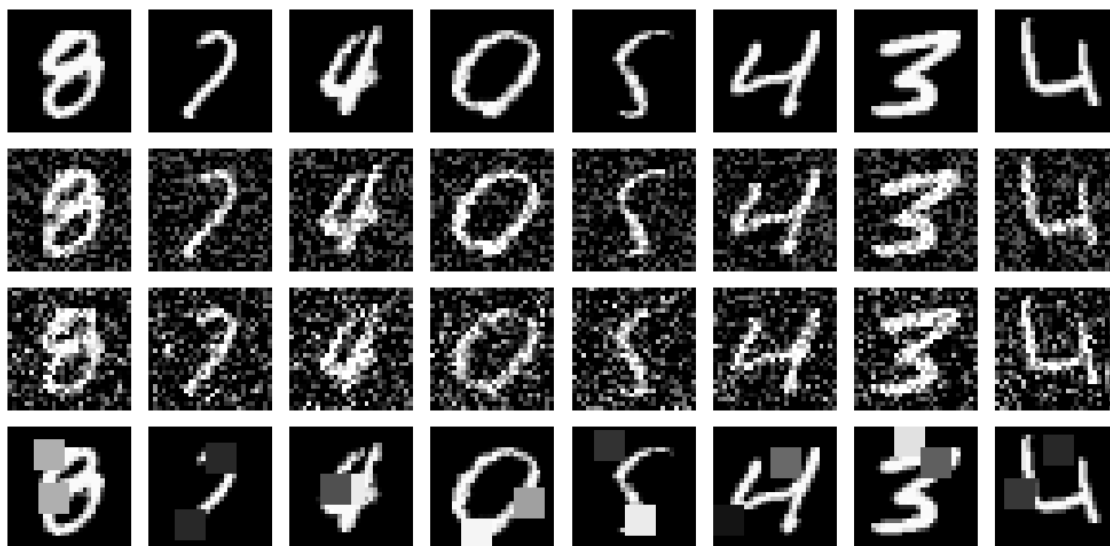


Рисунок 2 - Примеры различных вариантов зашумления изображений

Метрики качества восстановления

Для оценки качества восстановления используются численные метрики, рассчитываемые между восстановленным и исходным чистым изображением. Основной метрикой является среднеквадратичная ошибка MSE. Дополнительно рассчитываются PSNR и SSIM, позволяющие оценить качество изображения в более интерпретируемой форме.

Метрики рассчитываются отдельно для восстановления зашумленных изображений и для восстановления чистых изображений, что позволяет проверить как способность модели очищать шум, так и качество сжатия без дополнительного искажения входа.

Таблица 2 - Используемые метрики качества

Метрика	Назначение
MSE	Среднеквадратичная ошибка восстановления; меньшее значение соответствует лучшему восстановлению
PSNR	Пиковое отношение сигнал/шум; большее значение соответствует лучшему восстановлению
SSIM	Оценка структурного сходства изображения; большее значение соответствует лучшему восстановлению

MSE_GAIN_%	Относительное уменьшение MSE по сравнению с исходным зашумленным изображением
------------	---

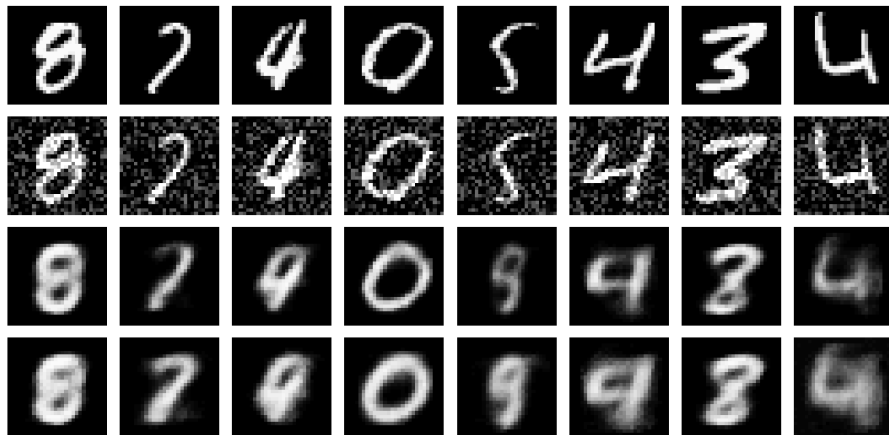
Исследование влияния типа шума

В первой серии экспериментов фиксировались архитектура автоэнкодера, размерность латентного пространства, число эпох и коэффициент обучения. Изменялся только тип зашумления: равномерный шум, нормальный шум и локальная порча пятнами.

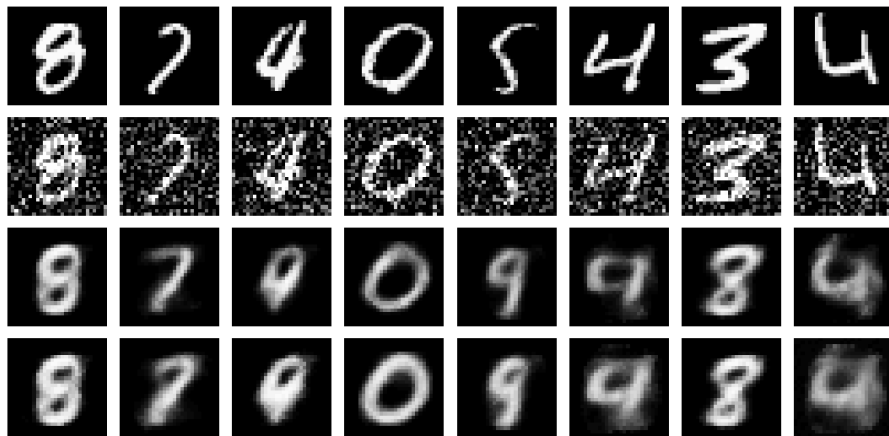
Таблица 3 - Результаты восстановления для разных типов шума

Тип шума	Noisy MSE	Denoise MSE	PSNR	SSIM	Вывод
Равномерный	0.03571	0.03073	15.12	0.793	Наименьшая итоговая ошибка среди трех типов шума
Нормальный	0.06244	0.03293	14.82	0.773	Ошибка относительно зашумленного входа снижена на 47.3%
Пятна	0.03670	0.03436	14.64	0.763	Самый сложный вариант по Denoise MSE; восстановление фрагментов хуже

Равномерный шум



Нормальный шум



Локальные пятна

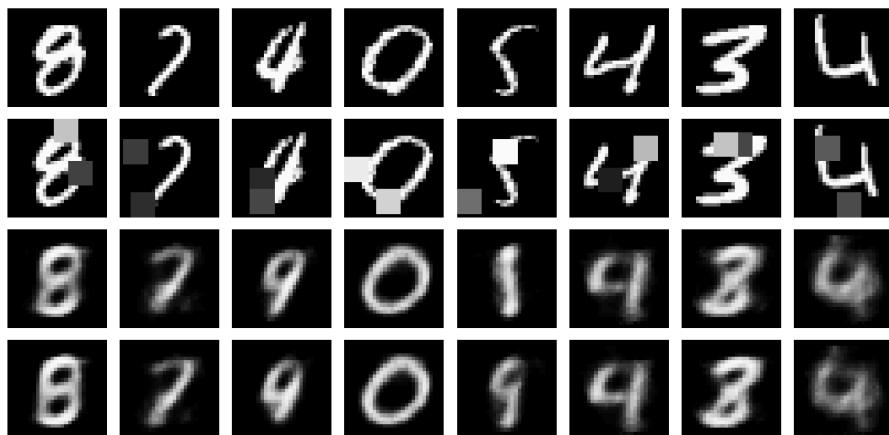


Рисунок 3 - Восстановление изображений при разных типах шума

По результатам эксперимента равномерный шум дал минимальную ошибку восстановления: $\text{Denoise MSE} = 0.03073$. Локальные пятна оказались

наиболее сложным вариантом, так как они повреждают целые участки цифры, а не только добавляют случайные отклонения к отдельным пикселям.

Исследование размерности латентного пространства

Во второй серии экспериментов проверялось влияние размерности пространства скрытых признаков. Остальные параметры обучения оставались фиксированными. Слишком малое латентное пространство должно приводить к потере деталей, поэтому сравнение проводилось для нескольких значений `latent_dim`.

Таблица 4 - Зависимость качества восстановления от размерности латентного пространства

latent_dim	Denoise MSE	PSNR	SSIM	Clean MSE	Вывод
8	0.03262	14.86	0.780	0.04064	Близко к лучшему результату, SSIM максимален
16	0.03249	14.88	0.773	0.04037	Лучший вариант по MSE и PSNR
32	0.03293	14.82	0.773	0.04194	Базовая конфигурация, немного уступает latent_dim = 16
64	0.03337	14.77	0.764	0.04244	Увеличение latent_dim не улучшило качество

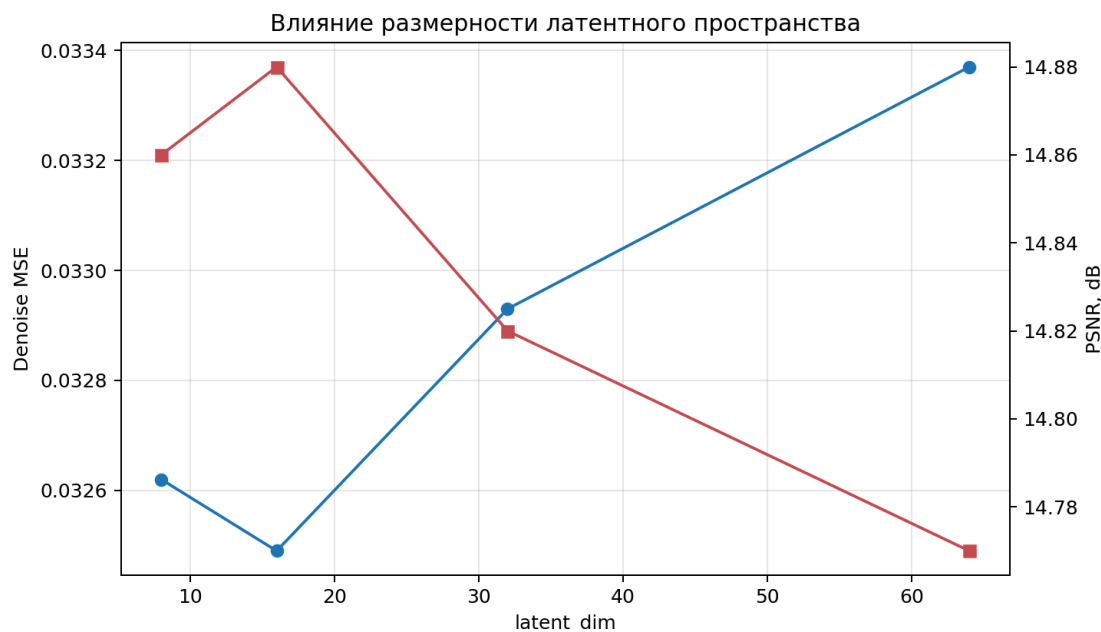


Рисунок 4 - Влияние размерности латентного пространства на качество восстановления

Лучшее значение Denoise MSE получено при latent_dim = 16. Дальнейшее увеличение размерности скрытого пространства не привело к улучшению качества в выбранной схеме обучения.

Исследование количества и ширины слоев кодера и декодера

В третьей серии экспериментов сравнивались несколько вариантов полносвязной архитектуры. Для корректного сравнения тип шума, размерность латентного пространства, число эпох и коэффициент обучения фиксировались.

Таблица 5 - Сравнение архитектур автоэнкодера

Архитектура	Скрытые слои	Параметры	Denoise MSE	PSNR	Вывод
shallow	(256)	419,120	0.02858	15.44	Лучший результат при минимальном числе параметров
medium	(512, 256)	1,083,696	0.03293	14.82	Базовый вариант, качество ниже shallow
deep	(512, 256, 128)	1,141,296	0.04486	13.48	Наихудший результат за 4 эпохи
wide	(768, 384)	1,821,744	0.02975	15.27	Второй результат, но параметров значительно больше

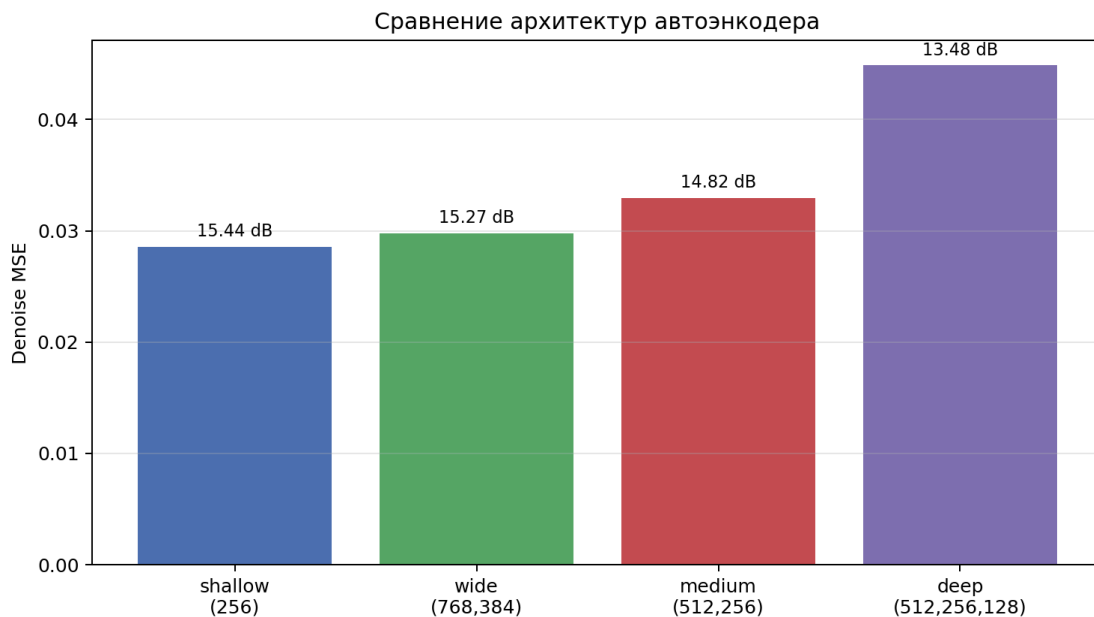


Рисунок 5 - Сравнение качества для разных архитектур автоэнкодера

В выбранных условиях лучший результат дала неглубокая архитектура shallow_256. Более широкая модель также показала хорошее качество, однако потребовала значительно больше параметров. Глубокая архитектура за то же число эпох обучилась хуже.

Исследование гиперпараметров обучения

В четвертой серии экспериментов оценивалось влияние числа эпох и коэффициента обучения. Для анализа фиксировались тип шума, архитектура и размерность латентного пространства.

Таблица 6 - Влияние числа эпох и learning rate

Эпохи	Learning rate	Denoise MSE	PSNR	Clean MSE	Вывод
4	0.0003	0.04910	13.09	0.06103	Слишком медленное обучение
4	0.001	0.03293	14.82	0.04194	Базовый устойчивый вариант
4	0.003	0.03101	15.08	0.03851	Лучше базового при 4 эпохах
8	0.001	0.02445	16.12	0.03330	Лучший результат среди гиперпараметров

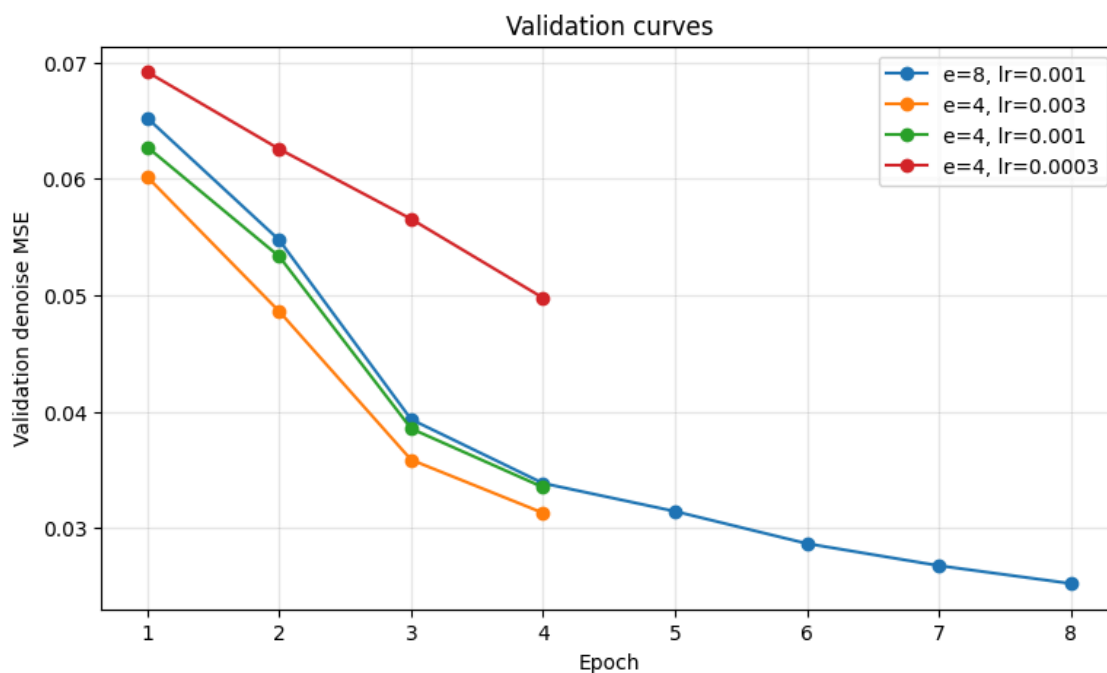


Рисунок 6 - Кривые обучения автоэнкодера

Минимальная ошибка достигнута при обучении в течение 8 эпох с learning rate = 0.001. При learning rate = 0.0003 модель обучалась слишком

медленно, а увеличение числа эпох заметно улучшило качество восстановления.

Оценка восстановления чистых и зашумленных изображений

После выбора лучшей конфигурации была выполнена отдельная проверка восстановления. Модель получает на вход как зашумленные изображения, так и чистые изображения. Это позволяет сравнить способность модели удалять шум и способность сжимать изображение без дополнительной порчи.

Таблица 7 - Качество восстановления чистых и зашумленных изображений

Тип входа	MSE	PSNR	Комментарий
Зашумленное изображение	0.02445	16.12	Лучшее восстановление для финальной конфигурации
Чистое изображение	0.03330	14.78	Модель сохраняет форму цифр, но часть деталей сглаживается при сжатии

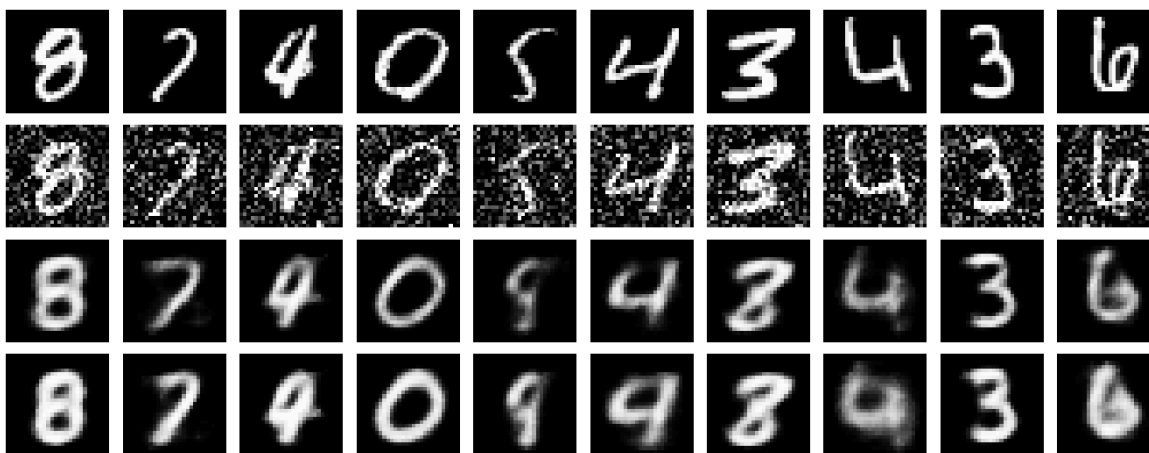


Рисунок 7 - Восстановление чистых и зашумленных изображений лучшей моделью

Визуально автоэнкодер восстанавливает основные контуры цифр и заметно уменьшает шум. При этом часть изображений становится сглаженной, что связано со сжатием в латентном пространстве и использованием полносвязной архитектуры.

Генерация изображений из латентного пространства

Для проверки возможности генерации новых изображений использовались случайные точки в латентном пространстве. Так как обычный автоэнкодер не является вероятностной генеративной моделью, точки

выбирались с учетом эмпирического распределения скрытых признаков, полученного на реальных изображениях.

Декодер получает случайно сгенерированные латентные векторы и преобразует их в изображения. Полученные результаты позволяют оценить, насколько непрерывным и пригодным для генерации оказалось пространство скрытых признаков.

Таблица 8 - Оценка генерации изображений из латентного пространства

Способ выбора точки	Масштаб разброса	Визуальная оценка результата
Эмпирическое распределение latent-векторов	1.0	Большинство образов похожи на цифры, но изображения заметно размыты
Расширенное эмпирическое распределение latent-векторов	1.8	Разнообразие выше, но артефактов и нечетких образов становится больше

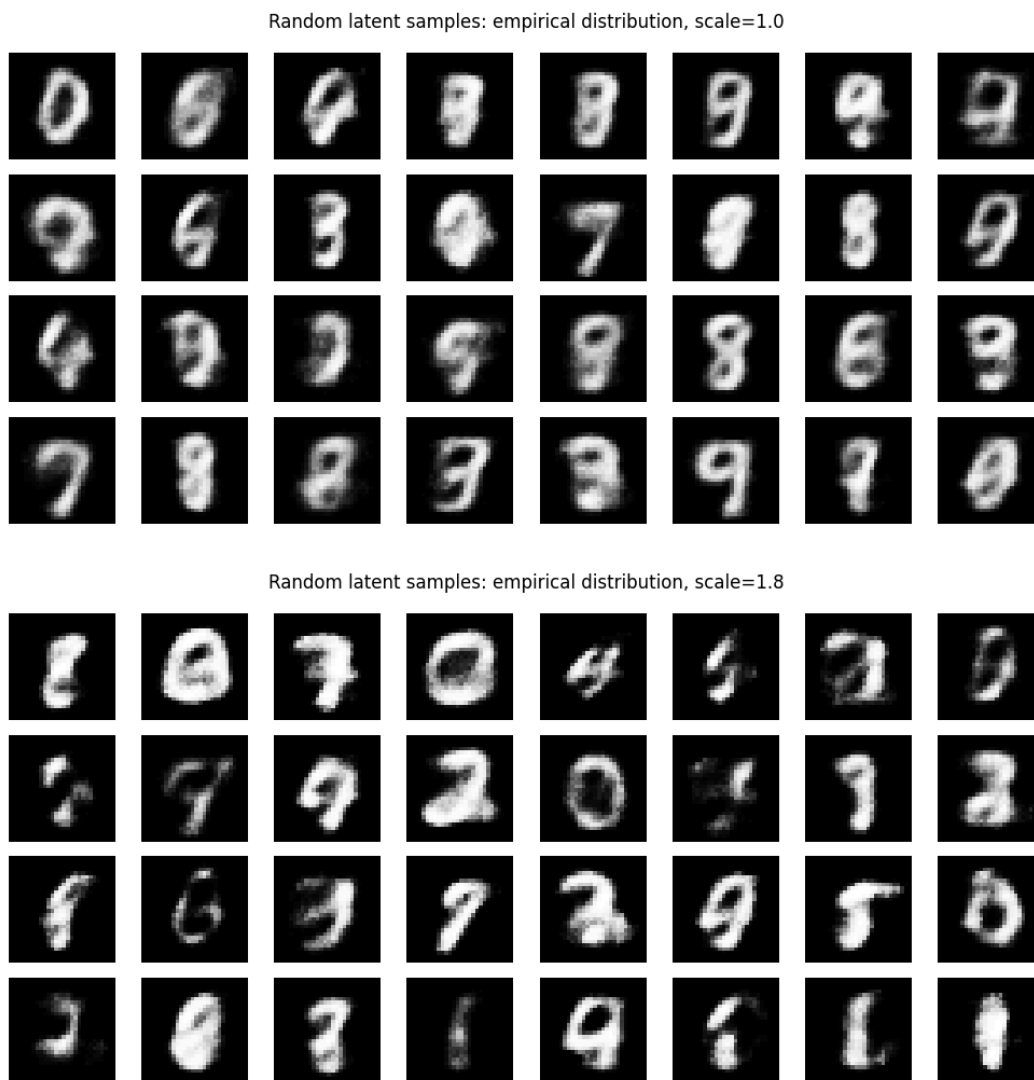


Рисунок 8 - Генерация изображений из случайных точек латентного пространства

Генерация из латентного пространства возможна: декодер строит изображения, похожие на рукописные цифры. Однако результат хуже, чем восстановление реальных изображений, потому что обычный автоэнкодер не регулирует латентное пространство как VAE или GAN.

Вывод

В ходе лабораторной работы был реализован автоэнкодер с полносвязными слоями для восстановления изображений и удаления шума. Были подготовлены функции равномерного, нормального и локального зашумления изображений, а также проведены эксперименты по сравнению качества восстановления при разных параметрах модели.

По результатам экспериментов среди типов шума минимальная итоговая ошибка восстановления была получена для равномерного шума, а наиболее сложным вариантом оказалась локальная порча пятнами. В серии экспериментов с размерностью скрытого пространства лучший результат показало `latent_dim = 16`, однако различия между значениями 8, 16 и 32 оказались небольшими.

Среди архитектур `encoder/decoder` лучшей по `Denoise MSE` стала `shallow_256` с одним скрытым слоем размера 256. В исследовании гиперпараметров лучший итоговый результат дала конфигурация с 8 эпохами и `learning rate = 0.001`.

Итоговая лучшая конфигурация: `gaussian sigma=0.35`, `hidden_dims=(512, 256)`, `latent_dim=32`, `epochs=8`, `lr=0.001`. Наименьшая ошибка восстановления: `Denoise MSE = 0.02445`, `PSNR = 16.12`. Генерация изображений из случайных точек латентного пространства возможна, но полученные изображения размыты и содержат артефакты, что ожидаемо для обычного автоэнкодера без вероятностной регуляризации.